Slot : D

19

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1407	**Course Title:**	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	P. Karpaga Shree	**Reg. No.:**	192424042
Section / Batch:	B.Tech (AI & DS)	**Date:**	16-07-2026

Code of Conduct (192424042)

I P. Karpaga Shree *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: P. Karpagashree

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 /4	3 /4	4 /4	4 /4	3 /4	91.25
2	4 /4	4 /4	3 /4	3 /4	4 /4	93.75
3	3 /4	3 /4	3 /4	3 /4	4 /4	88.75
4	4 /4	4 /4	3 /4	3 /4	3 /4	85
5	3 /4	3 /4	4 /4	4 /4	4 /4	90
OVERALL RUBRICS VALUE						
	/4	/4	/4	/4	/4	89.75
	/20	/20	/25	/20	/15	/100

N.Dj

Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression P = (a * b) + (b * c) * 2 through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a)$(\varepsilon + aa\ )$ b)$(a\ \ +b\ )\ abb$ c)$(a\ \ b\ )\ aba(a\ \ +b\ )$ d)$(a + ab)$ e) $(aba)\ \ +(a\ \ b\ )$

3	i)Find the number of tokens in the following C statement is
	main( )
	{
	int a=10;
	char b ="abc";
	in t c = 30;
	ch ar d = "xyz";
	in /* comment */ t m = 40.5;
	}
	ii)Identify the lexical errors in the following C statement is
	void main()
	{
	int x=10, y=20;
	char * a;
	a= &x;
	x= 1xab;
	}

4	**Find a regular expression corresponding to each of the following subsets of** $\{0,1\}$: i) The language of all strings that have an even number of 0's. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0's and the number of 1's

are odd.

iv) The language of all strings that start with 1 and end with 0.

v) The language of all strings that do not contain consecutive 1's.

5 **Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:**

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, ;, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string: int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }

Questions:

a) How many tokens are there in the given input string?

b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).

c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

ASSESSment 1 - Analytical problem solving

① Trace the processing of the expression

Expression

$P = (a * b) + (b * c) * 2$

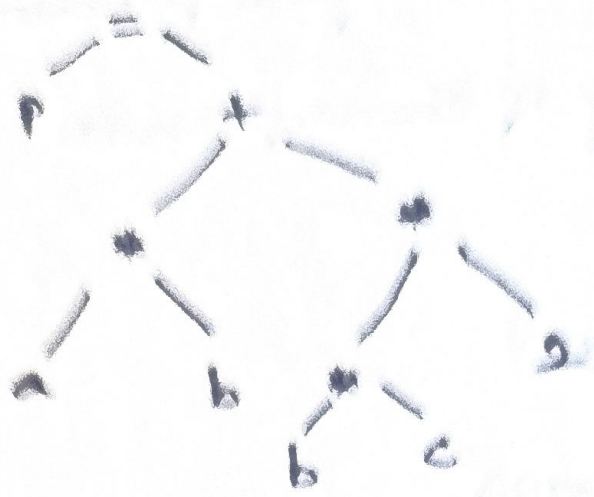
a) Lexical analysis (TOKENS)

Lexeme	→	TOKENS
P		Identifier
=		Assignment operator
(		Left parenthesis
a		Identifier
*		Multiplication operator
b		Identifier
)		Right parenthesis
+		Addition operator
(		Left parenthesis
b		Identifier
*		Multiplication operator
c		Identifier
)		Right parenthesis
*		Multiplication operator
2		Integer constant

Total TOKENS = 15

b) Syntax Tree

↳



c) Intermediate code (Three address code)

$t1 = a * b$

$t2 = b * c$

$t3 = t2 * 2$

$t4 = t1 + t3$

d) optimized Intermediate code

$t1 = a * b$

$t2 = b * c$

$t2 = t2 << 1$ (multiply by 2 using left shift)

$p = t1 + t2$

(optimization : strength Reduction and fewer temporary variables.)

e) Target code (Example)

```
LOAD R1, a
MUL R1, b
LOAD R2, b
MUL R2, c
SHL R2, 1
ADD R1, R2
STORE P, R1
```

significance:

* Executes directly on the processor.
* optimized for faster execution.
* uses fewer instructions and memory.

(2) construct the Language (write any five strings)

a) Regular Expression: $(aa^*)^*$

ε
a
aa
aaa
aaaa

b) $(b^*)^* abb$

since $(b=b^*$, Language: all strings of zero of more b's followed by "abb".

abb
babb
bbabb
bbbabb
bbbbabb

c) $(a^1)^1 + b^{1*})^{1*}$

Language: all strings that contain the substring "aba".

aba
abaa
abab
aaba
baba

d) $(ab)^*$

Language: zero or more repetitions of "ab".

ε
ab
abab
ababab
abababab

(2) $(aba)^* + (a^*)^*$

This is the union of

* zero or more repetitions of "aba", or
* Any string of a's followed by b's (including the empty string).

ε
aba
abaaba
aaabbb
aab

(3) a) find the number of tokens in the following c statement.

```
main()
{
int a = 10;
char b = "abc";
int c = 30;
char d = "xyz";
int m = 40.5;
}
```

TOKens:

S.NO	TOKen	Type
1	main	identifier
2	(	left parenthesis
3	)	Right parenthesis

4	{	Left Brace
5	int	Keyword
6	a	Identifier
7	=	Assignment operator
8	10	Integer constant
9	;	Seperator
10	char	Keyword
11	b	Identifier
12	=	Assignment operator
13	"abc"	String Literal
14	;	seperator
15	in	Identifier (error)
16	t	Identifier
17	c	Identifier
18	=	Assignment operator
19	30	Integer constant
20	;	seperator
21	ch	Identifier
22	ar	Identifier
23	d	Identifier
24	=	Assignment operator
25	"xyz"	String Literal
26	;	seperator
27	in	Identifier
28	t	Identifier
29	m	Identifier
30	=	Assignment operator

	31	40.5	floating-point constant
	32	;	Separator
	33	}	Right brace

Total number of tokens = 33

ii) Identify the lexical errors

```
void main()
{
    int x = 0, y = 20
    char *a;
    a = &x;
    x = 1xab;
}
```

lexical error:

x = 1xab;

is invalid

Reason:

* 1xab is not a valid integer or hexadecimal constant.
* In C, a hexadecimal number must begin with 0x or 0X.

Correct statement:

x = 0XAB;

Other statements:

```
int x = 0, y = 20;
char *a;
a = &x;
```

These are lexically valid (although a = &x; gives a types mismatch warning because a is a char* and &x is an int*. This is not a lexical error, it is a type / semantic issue).

4) Find a Regular expression corresponding to each of the following subsets of {0,1}

i) The language of all strings that have an even number of 0's.

Regular expression :

$1^* (01^* 01^*)^*$

Examples :

ϵ

11

00

1001

0101

ii) The language of all strings that contain atleast one 1.

Regular expression :

$(0+1)^* 1 (0+1)^*$

Examples :

1

10

01

101

111

iii) The language of all strings in which both the number of 0's and the number of 0's and the number of 1's are odd.

Regular expression :

$(01+10)(00+11)^*(01+10)$

Examples :

01

10

0001

0111

1100

iv) The language of all strings that start with 1 and end with 0.

Regular expression:

1(0+1)* 0

examples:

10

110

100

1010

111110

v) The language of all strings that do not contain consecutive 1's.

Regular expression:

(0+10)* 1?

Examples:

ε

0

1

01010

1010.

(5) Given input string.

```
int var123 =12345;
float var456 =123.45;
if (var123==var456)
{
    return;
}
```

a) How many tokens are there in the given input string?

The input string contains 20 tokens.

Token	Type
int	Keyword
var123	Identifier
=	operator
10345	literal
;	Separator
float	Keyword
var456	Identifier
= =	operator
125.45	floating-point literal
;	Separator
if	keyword
(	Separator
var123	Identifier
)	Separator
{	Separator
var456	Identifier
= =	operator
return	keyword
;	Separator
}	Separator

Total number of tokens = 20

b) provide a list of tokens categorized by their type.

Keywords :

* int
* float
* if
* return

Separators :

* (
*)
* {
* }
* ;
* ;
* ;

operators :

* =
* =
* ==

Identifiers :

* var 123
* var 456
* var 123
* var 456

Integer Literal

* 12345

Floating-point Literal

* 123.45

c) Assuming a Finite state machine is used for Lexical analysis, FSM State Transition

Approximately 62 state transitions.

Reason: The FSM reads each character and changes states to recognize keywords, identifiers, operators, separators and literals. Hence 62 state transitions are required.

Keywords = 4

Identifiers = 24

operators = 4

separators = 7

Integer Literal = 5

Floating-point literal = 6

Total Estimated FSM state transitions = 62.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation